

Sorting - 2

TABLE OF CONTENTS

1. Pivot Partition
2. Quick Sort
3. Comparator Problems



PSP - 69.67



Notes

Contest Prep

Topic	Sub-topics	Importance
Arrays	Prefix, Carry Fwd, Contn, Slid Wpn, 2D array, Pater	V.V.Pmp
Bit Manipu-lation	check bit, set bit, toggle, XOR, Bit Manipulation 2	V.V.Pmp
Recursion & Math	3 magic steps, Modulo, GCD	V.Pmp
	len priority	
Hashing & Sorting	Count Pair sum, Substring Subarray with sum=0	V.Pmp



Given an integer array, consider first elements as pivot, rearrange the elements such that for all i :

- If $A[i] \leq p$ then it should be present on left side.
- If $A[i] > p$ then it should be present on right side.

pivot is the first element

return the pivot Index

$\leq$	P	$>$
--------	---	-----

arr =

54	26	93	17	77	31	44	55	20
0	1	2	3	4	5	6	7	8

After partition =

26	17	31	44	20	54	55	77	93
0	1	2	3	4	5	6	7	8

Index = 5

Compare, is $54 > 26$, no then 26 will come before 54. If $93 > 54 \rightarrow$ yes then it go after 54 at the end

Quiz 1

arr =

10	13	7	8	25	20	23	5
0	1	2	3	4	5	6	7

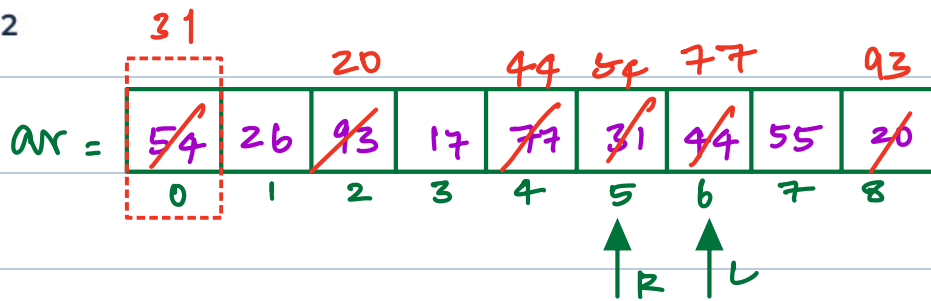
After partition =

7	8	5
0	1	2

23	20	25	13
4	5	6	7

Partition the Array

① Two pointer approach (L, R)



```
int partition ( int arr, int L, int R ) {
```

my 1st element would be pivot

```
    pivot = arr[L]; first = L;
```

```
    L = L + 1;
```

my left pointer would point to next element of pivot

```
    while ( L <= R ) {
```

```
        if ( arr[L] <= pivot ) {
```

```
            L ++;
```

```
        else if ( arr[R] > pivot ) {
```

```
            R --;
```

if right element is greater than pivot, it should be there at right side only --> so no change

```
        else {
```

```
            swap ( arr[L], arr[R] )
```

for L = 2 and R = 8, these should be present at their respective sides so we will mutually swap them

```
        swap ( arr[first], arr[R] );
```

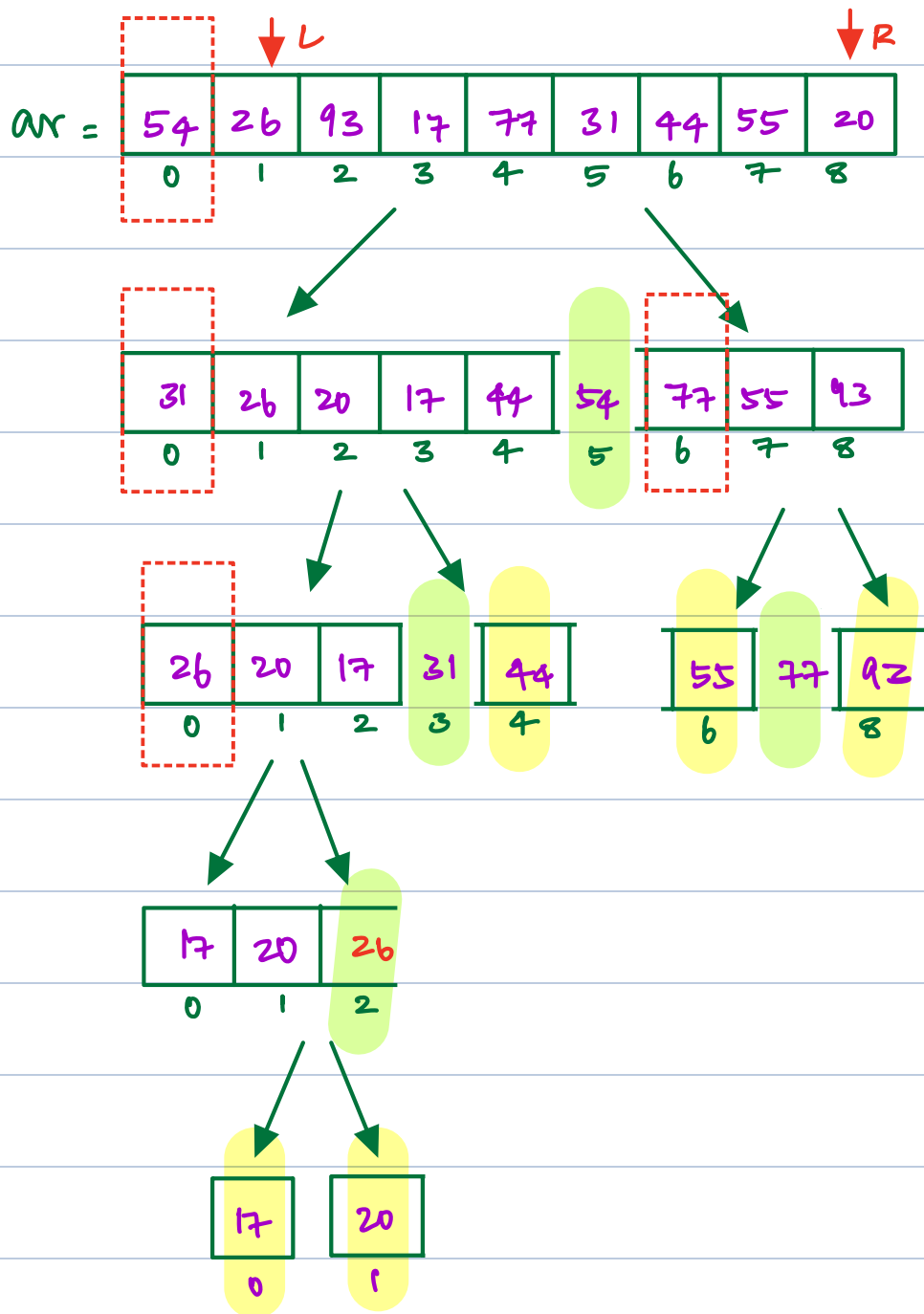
```
        return R;
```

since first will have value of index = 0

T.C = $O(N)$ S.C = $O(1)$



Quick Sort



pivot = 54

mid = 5

arr =

17	20	26	31	44	54	55	77	93
0	1	2	3	4	5	6	7	8



Assumption:

Quick sort (arr, s, e)

Main logic:

- ① . p = pivot Idx (arr, s, e)
- ② Quick sort (arr, s, p-1)
- ③ Quick sort (arr, p+1, e)

Base Condition

If (s >= e) return

</> Code

```
void Quick sort ( int arr , int 0/s , int n-1/e ) {
```

```
    if ( s >= e ) return;
```

```
    int p = partition ( arr , s , e );
```

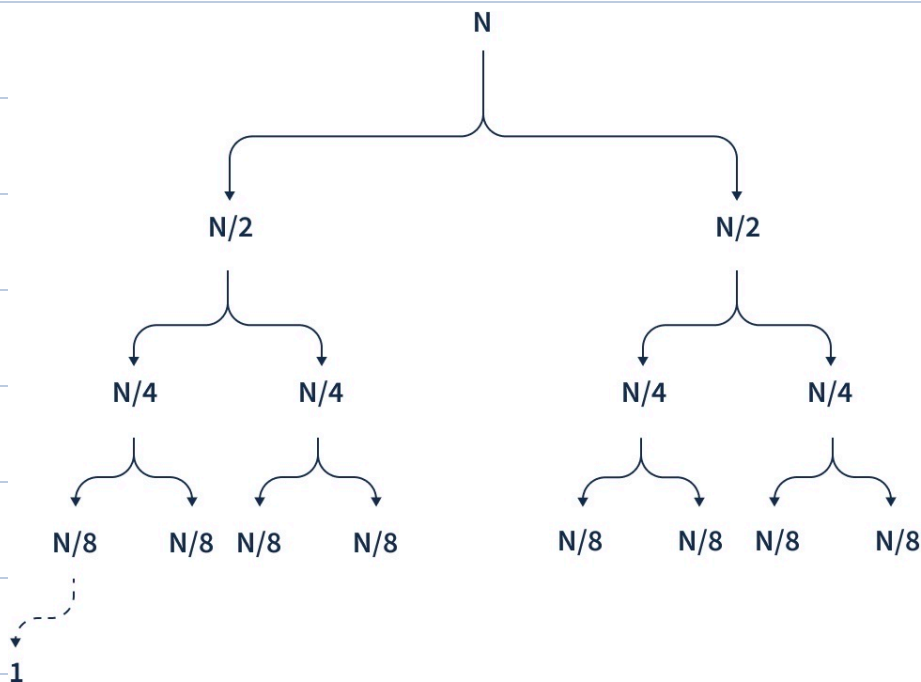
```
    Quick sort ( arr , s , p-1 );
```

```
    Quick sort ( arr , p+1 , e );
```

```
}
```



Best - Case

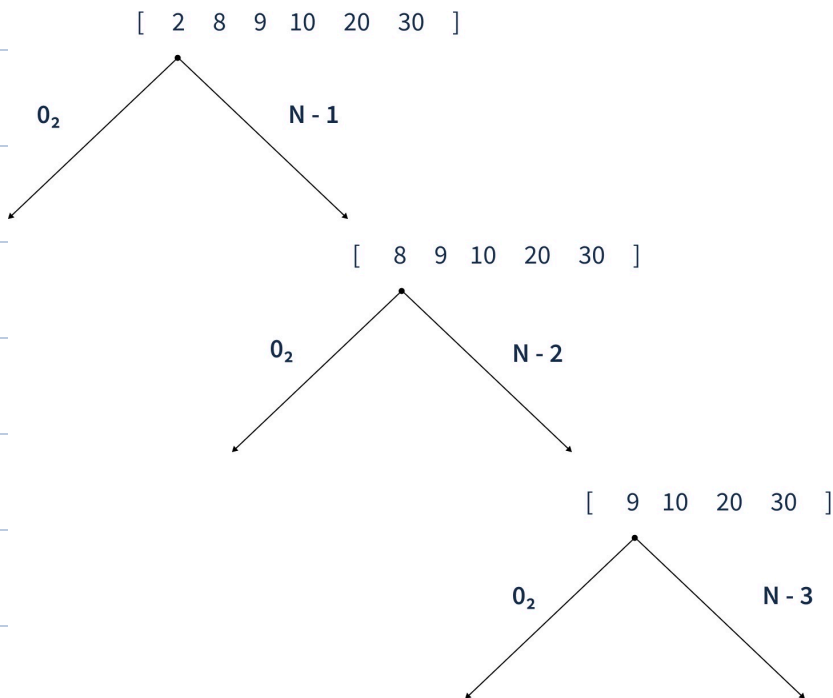


$$T.C = \frac{N * \log N}{\text{no. of steps}}$$

Instruction per step

$$S.C = \frac{O(\log N)}{\text{stack space}}$$

Worst - Case



$$T.C = \frac{O(N * N)}{\text{no. of steps}}$$

Instruction per step

$$S.C = O(N)$$



Randomised Quick Sort

Given N unique elements

→ Probability a random no. is minimum?

$$\frac{1}{N}$$

→ Remaining $N-1$ elements, Probability that a random no. is minimum?

$$\frac{1}{N-1}$$

→ Remaining $N-2$ elements, Probability that a random no. is minimum?

$$\frac{1}{N-2}$$

→ Remaining $N-3$ elements, Probability that a random no. is minimum?

$$\frac{1}{N-3}$$

Break : 10:15

↳ 10:25 pm

Total Probability

$$\frac{1}{N} * \frac{1}{N-1} * \frac{1}{N-2} * \dots * \frac{1}{1} = \frac{1}{N!}$$



Custom Comparator

Inbuilt sort function has a default comparison based on value of the element. We can however have our own custom comparison

Java, Python, JS, Ruby

Put `compare (val1, val2)`

① If we want `val1` to come before `val2`

return -ve

② If we want `val2` to come before `val1`

return +ve

③ If `val1 == val2` return 0.

C++

boolean `compare (val1, val2)`

① If we want `val1` to come before `val2`

return True

② If we want `val2` to come before `val1`

return False



Question

Given $\text{arr}[N]$, sort the data in ascending order of count of factors. If count of factors are equal, then sort the elements on the basis of their values.

$\text{arr} =$	9	3	10	6	4
$\text{factors} =$	3	2	4	4	3
$\text{ans} =$	3	4	9	6	10
	0	1	2	3	4

Ans 2

$\text{arr} =$	10	4	5	13	1
$\text{factors} =$	4	3	2	2	1
$\text{ans} =$	1	5	12	4	10

#pseudo code

```

Put count factors ( Put N ) {
    Put ans = 0;
    for ( Put P = 1; P * P <= N; P++ ) {
        If ( N % P == 0 ) {
            If ( N / P == P ) count += 1;
            else count += 2;
        }
    }
    return count;
}

```

Java

Collection.sort (A, new Comparator < Integer > () {

@ Override

public int compare (Integer v₁, Integer v₂) {

int f₁ = count_factors (v₁);

int f₂ = count_factors (v₂);

if (f₁ < f₂) return -1;

else if (f₁ > f₂) return 1;

else {

return v₁ - v₂;

}

}

});

C++

```
bool compare(int val1, int val2)
{
    int cnt_x = count_factors(x);
    int cnt_y = count_factors(y);
    if(factors(val1) == factors(val2))
    {
        if(val1<val2)
        {
            return true;
        }
        return false;
    }
    else if(factors(val1)<factors(val2))
    {
        return true;
    }
    return false;
}

vector<int> solve(vector<int> A) {
    sort(A.begin() , A.end() , compare);
    return A;
}
```

Python

```
import functools

def compare(v1, v2):
    if(factors(v1) == factors(v2)):
        if(v1<v2):
            return -1;
        if(v2<v1):
            return 1;
        else
            return 0;
    elif (factors(v1)<factors(v2)):
        return -1;
    else
        return 1;

class Solution:
    def solve(self, A):
        A = sorted(
            A,
            key = functools.cmp_to_key(compare)
        )
        return A
```

Question

Given a list of non-negative integers `nums`, arrange them such that they form the largest number and return it.

`arr =`

10	2
----	---

`ans =`

2	10
---	----

`arr =`

3	30	34	5	9
---	----	----	---	---

`ans =`

9	5	34	3	30
---	---	----	---	----

Quiz

`arr =`

10	5	2	8	200
----	---	---	---	-----

`ans =`

8	5	2	200	10
---	---	---	-----	----

Approach

$x = 9 \rightarrow "9"$ $y = 100 \rightarrow "100"$

$str(x) + str(y) > str(y) + str(x)$

x comes first

```
def compare (int x, int y):
```

```
    string x-s = x.toString();
```

```
    string y-s = y.toString();
```

```
    if (x-s + y-s > y-s + x-s) return -1;
```

```
    else if (y-s + x-s > x-s + y-s) return 1;
```

```
    else return 0;
```

```
}
```

C++

```
bool compare(int a, int b) {
    return to_string(a) + to_string(b) >
to_string(b) + to_string(a);
}

string largestNumber(vector<int> &A) {
    sort(A.begin(), A.end(), compare);
    string ans = "";
    for (auto &x : A)
        ans += to_string(x);
    if (ans[0] == '0') return "0";
    return ans;
}
```

Python

```
from functools import cmp_to_key

class Solution:
    # @param A : list of integers
    # @return a strings
    def largestNumber(self, A):
        def cmp_func(x, y):
            if x + y > y + x:
                return -1
            elif x == y:
                return 0
            else:
                return 1

        nums = [str(num) for num in A]
        nums.sort(key = cmp_to_key(cmp_func))
        if nums[0] == '0':
            return '0'
        return ''.join(nums)
```